42. Design Ticket Booking System

Key Terms:

Term	Full Name	What it does
Virtual Queue	Waiting room	Controlled admission to prevent overload
Hold	Temporary reservation	Tickets reserved during checkout (10 min)
FCFS	First-Come-First-Served	Queue-based fairness
Overselling	Selling more than available	Must be prevented with atomic operations
CAPTCHA	Bot challenge	Distinguishes humans from automated bots
Saga Pattern	Distributed transaction	Sequence of steps with compensations
Scalping	Ticket resale for profit	Prevented with purchase limits

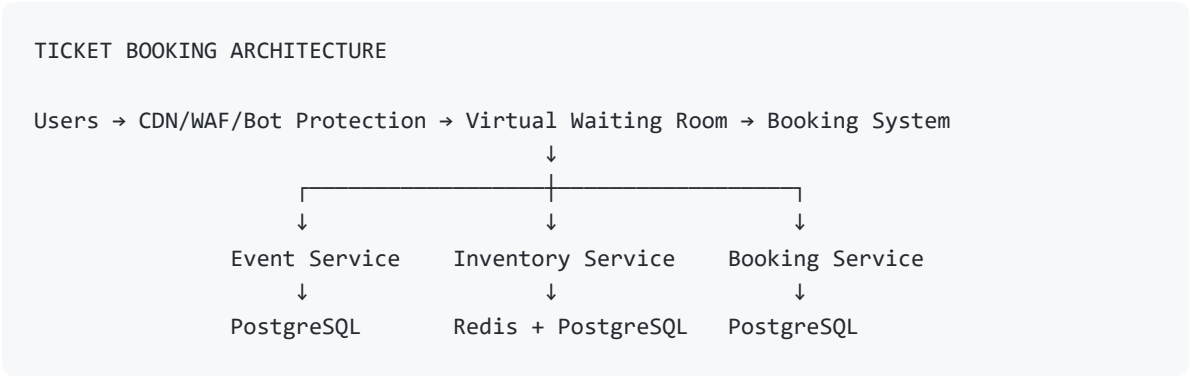
Problem Statement

Design a ticket booking system like Ticketmaster or Eventbrite that handles high-concurrency purchases for events with limited inventory.

Clarifying Questions to Ask

1. **Scale**: Concurrent users during hot sales? (100K-1M simultaneous?)
2. **Inventory**: Tickets per event? Seat selection required?
3. **Fairness**: First-come-first-served (FCFS) or lottery?
4. **Hold time**: How long to hold tickets during checkout? (5-15 min?)
5. **Pricing**: Fixed, tiered, or dynamic pricing?
6. **Payment**: In-system or external payment provider?
7. **Fraud**: Bot detection and prevention requirements?
8. **Resale**: Support ticket transfers or resale marketplace?

High-Level Design



API Design

Event APIs

Method	Endpoint	Description
`GET`	`/api/v1/events`	List events
`GET`	`/api/v1/events/{eventId}`	Get event details
`GET`	`/api/v1/events/{eventId}/sections`	Get seating sections and availability

Booking APIs

Method	Endpoint	Description
`POST`	`/api/v1/bookings/hold`	Hold seats (starts 10-min timer)
`POST`	`/api/v1/bookings/confirm`	Confirm booking and process payment
`DELETE`	`/api/v1/bookings/{bookingId}`	Cancel booking
`GET`	`/api/v1/bookings/{bookingId}`	Get booking details

Hold Seats Request

```
POST /api/v1/bookings/hold
{
  "event_id": "evt_123",
  "seats": ["A-12", "A-13"],
  "user_id": "user_456",
  "session_token": "queue_token_789"  // from waiting room
}

Response: 200 OK
{
  "hold_id": "hold_001",
```

```
    "seats": ["A-12", "A-13"],
    "expires_at": "2025-03-15T10:10:00Z",
    "total_price": 150.00
  }
```

Waiting Room APIs

Method	Endpoint	Description
`POST`	`/api/v1/queue/join`	Join virtual waiting room
`GET`	`/api/v1/queue/position`	Get queue position

Virtual Waiting Room

```
REDIS SORTED SET QUEUE:
┌──────────────────────────────────────────────────────────────────────┐
│  Key: queue:event:{event_id}                                         │
│  Members: user tokens                                                │
│  Scores: join timestamp                                              │
│                                                                      │
│  # User joins queue                                                  │
│  ZADD queue:event:123 1705312200 "token_abc123"                      │
│                                                                      │
│  # Get position                                                      │
│  ZRANK queue:event:123 "token_abc123" → 45230                        │
│                                                                      │
│  # Admit next 100 users                                              │
│  ZPOPMIN queue:event:123 100                                         │
│                                                                      │
│  Admission rate = System capacity / Avg checkout time                │
│  Example: 1000 concurrent / 3 min = 333 users/minute                 │
└──────────────────────────────────────────────────────────────────────┘
```

Inventory Management

```
ATOMIC RESERVATION (Lua script):
┌──────────────────────────────────────────────────────────────────────┐
│  local available = redis.call('GET', KEYS[1])                        │
│  local requested = tonumber(ARGV[1])                                 │
│                                                                      │
│  if tonumber(available) >= requested then                            │
│      redis.call('DECRBY', KEYS[1], requested)                        │
│      redis.call('INCRBY', KEYS[2], requested)  -- reserved           │
│      return 1  -- Success                                            │
│  else                                                                │
│      return 0  -- Not enough                                         │
```

```
│  end                                                                    │
└─────────────────────────────────────────────────────────────────────────┘

HOLD PATTERN:
  1. User selects tickets → Reserve in Redis (atomic)
  2. Create hold record with 10-min expiry
  3. On payment success → Convert to booking, update sold count
  4. On expiry → Release back to available
```

Booking Flow

```
BOOKING STATE MACHINE:

  BROWSING → HELD (10 min timer) → CONFIRMED
                    ↓
             EXPIRED / FAILED
             (Release inventory)

HOLD IMPLEMENTATION:
  1. INSERT hold record (PostgreSQL)
  2. DECRBY available, INCRBY reserved (Redis atomic)
  3. SET hold:id "" EX 600 (10 min expiry trigger)
  4. On expire → release inventory, delete hold
```

Database Schema

```sql
  CREATE TABLE events (
      id BIGINT PRIMARY KEY,
      name VARCHAR(255),
      venue_id BIGINT,
      date TIMESTAMP,
      status VARCHAR(20),  -- on_sale, sold_out
      total_tickets INT
  );

  CREATE TABLE bookings (
      id BIGINT PRIMARY KEY,
      event_id BIGINT,
      user_id BIGINT,
      quantity INT,
      total_amount DECIMAL(10,2),
      status VARCHAR(20),  -- confirmed, cancelled
      created_at TIMESTAMP
  );

  CREATE TABLE holds (
      id BIGINT PRIMARY KEY,
      event_id BIGINT,
      user_id BIGINT,
```

```
    quantity INT,
    expires_at TIMESTAMP
);
```

Bot Prevention

```
ANTI-BOT LAYERS:
  1. Rate limiting: 60 req/min per IP
  2. CAPTCHA: reCAPTCHA v3 (invisible) + v2 challenge if suspicious
  3. Device fingerprinting: Block known bot signatures
  4. Behavior analysis: Detect inhuman speed/patterns
  5. Purchase limits: Max 4 tickets per user per event
```

Key Components

Component	Purpose	Technology
Queue Service	Virtual waiting room	Redis Sorted Set
Inventory Service	Track availability	Redis + PostgreSQL
Booking Service	Purchase orchestration	Saga pattern
Payment Service	Process payments	Stripe
Bot Protection	Prevent abuse	Cloudflare + reCAPTCHA

Scalability Considerations

- **Queue handles spikes**: 100K users wait, 1K process at a time
- **Redis for hot inventory**: Atomic operations prevent overselling
- **Horizontal scaling**: Stateless services behind load balancer
- **CDN caching**: Event pages cached at edge
- **Async notifications**: Emails via queue

Trade-offs

Decision	Option A	Option B	Recommendation
Queue	In-memory	Redis	**Redis** - persistent, distributed
Inventory	Database	Redis	**Redis** - atomic, fast

Hold duration	5 min	15 min	**10 min** - balance UX vs lock
Fairness	FCFS	Lottery	**FCFS + queue** - transparent

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Virtual waiting room queue, atomic inventory counters, hold expiration	Tech Guide
PostgreSQL	Persistent storage for events, bookings, and holds	Tech Guide
Saga Pattern	Orchestrating booking flow with compensations for failures	Tech Guide

How to Explain It Well

> **Opening**: "Ticket booking handles extreme traffic spikes with limited inventory. I'll use a virtual queue to smooth demand and Redis atomic operations to prevent overselling."

> **Key talking points**:
>
> - "Virtual queue: Redis sorted set, admit users at controlled rate"
> - "Redis atomic decrement prevents overselling under high concurrency"
> - "Hold pattern: reserve tickets for 10 minutes during checkout"
> - "Expired holds auto-release inventory back to pool"
> - "Bot protection: rate limits, CAPTCHA, fingerprinting"
> - "Max 4 tickets per user to prevent scalping"

Common Follow-up Questions

1. **How do you handle abandoned checkouts?**
    - Hold expires after 10 minutes
    - Background job releases inventory
    - Reminder email at 5 minutes
2. **How do you prevent scalpers?**

- Purchase limits (4 per user)
- Verified identity required
- Non-transferable tickets option

3. **What if Redis goes down?**

- Redis Cluster with replicas
- Fallback to PostgreSQL
- Queue pauses until recovery